

Keys of pythons:

Topic	Syntax	Real-Time Architectural Use	Interview Cross-Question
List	[1, 2]	Storing a batch of record IDs for sequential processing.	"Why is pop(0) inefficient in a long list?" (Ans: $O(n)$ time complexity as all elements must shift).
Tuple	(1, 2)	Database Credentials or Schema Definitions. Immutability ensures configurations aren't changed at runtime.	"Can a tuple contain a mutable object?" (Ans: Yes, a list inside a tuple can be modified, but the tuple's reference cannot).
Set	{1, 2}	De-duplication of User IDs during ingestion or fast membership testing ($O(1)$ lookup).	"How does a Set handle lookups so fast?" (Ans: Hashing algorithm).

Dict	<code>{"a": 1}</code>	JSON Parsing and mapping Source Columns to Target Columns in a metadata-driven pipeline.	"What happens if two keys hash to the same value?" (Ans: Hash collision; Python handles this via Open Addressing).
-------------	-----------------------	---	--

1. Difference Between List, Tuple, Set, Dictionary

Type	Syntax	Ordered	Mutable	Duplicate Allowed
List	<code>[]</code>	✓	✓	✓
Tuple	<code>()</code>	✓	✗	✓
Set	<code>{}</code>	✗	✓	✗
Dictionary	<code>{"key":value}</code>	✓	✓	Keys ✗

Empty_

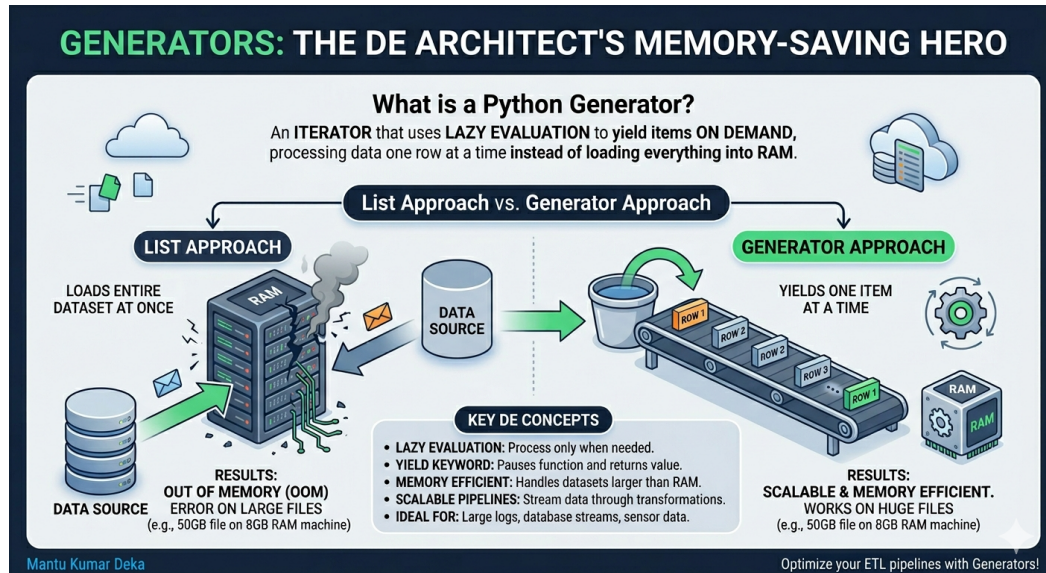
```
a=[]
b=()
c={}
d=set()
```

With one item:

```
a = [1]
b = (1,)
c = {1}
d = {"x":1}
```

What is generator Generator_

A generator is a **memory-efficient iterator** that uses **lazy evaluation** to yield one item at a time on demand rather than storing an entire dataset in RAM.



Feature	List / Collection	Generator
Storage	Entirely in RAM	One item at a time
Speed	Faster if you need to access items multiple times	Faster to <i>start</i> (low latency)
Access	Indexing (e.g., data[5])	Sequential only (next())
Persistence	Can be reused	Exhausted after one use

Topic	Syntax	Real-Time Architectural Use	Interview Cross-Question
-------	--------	-----------------------------	--------------------------

Generator	yield	Streaming large S3 files. Loading 10GB file into 8GB RAM without crashing (OOM).	"Difference between yield and return?" (Ans: Yield saves state and pauses; return destroys state).
Big File Read	for line in f:	Processing logs or CSVs line-by-line to maintain a constant memory footprint.	"How do you read a file in chunks of 1000 lines?" (Ans: Use <code>itertools.islice</code>).
Shallow vs Deep	<code>copy.deepcopy()</code>	Cloning a complex configuration object before modifying it for a sub-pipeline.	"Which one is faster?" (Ans: Shallow, as it only copies references, not data).

What are Decorators?

A decorator is a function that takes another function and extends its behavior without explicitly modifying its source code. Think of it like a **wrapper** or a protective layer you put around a core process.

-A decorator is essentially a "function within a function."

Why Decorators Matter in Data Engineering

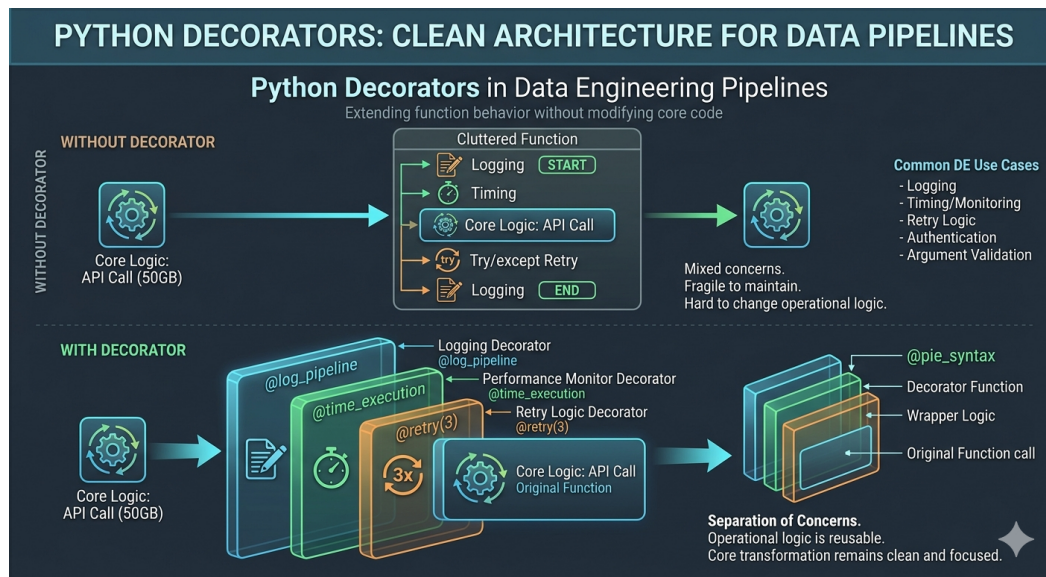
In production pipelines, we often need to add logic that isn't part of the "core business logic." Instead of cluttering your transformation code with logs or timers, you use a decorator.

Common use cases include:

- **Logging:** Automatically log when a pipeline stage starts and finishes.

- **Timing:** Measure how long a specific data load takes.
- **Retry Logic:** If an API call fails, automatically try again 3 times before crashing.
- **Authentication:** Check credentials before allowing a data write.

Generators are about **Memory Efficiency** (how we handle data).
Decorators are about **Code Efficiency and Maintenance** (how we handle logic).



Topic	Syntax	Real-Time Architectural Use	Interview Cross-Question
Lambda	lambda x: x+1	Anonymous cleanup logic inside a map() or Spark udf.	"Why avoid complex Lambdas?" (Ans: Hard to debug/test; use a named function for complex logic).

Map/Filter	map(func, iter)	ETL Cleansing: Map to cast types; Filter to remove nulls or "bad" records.	"Are map/filter lazy in Python 3?" (Ans: Yes, they return iterators, not lists).
Comprehension	[x for x in i]	Fast creation of lookup tables or simple row transformations.	"When is a comprehension worse than a loop?" (Ans: When logic is nested/complex, it ruins readability).

Topic	Syntax	Real-Time Architectural Use	Interview Cross-Question
Decorator	@wrapper	Audit Logs & Retries: Automatically timing an ETL job or retrying an API call 3 times.	"How do you pass arguments to a decorator?" (Ans: Create a triple-nested function).
Context Mgr	with open...	Resource Safety: Ensuring DB connections and File handles close even if the script crashes.	"How do you create a custom one?" (Ans: Define __enter__ and __exit__ methods).

*args/ **kwargs	def f(*a, **k)	Creating Generic Wrappers or Factory functions that handle varying source schemas.	"What is the order of arguments?" (Ans: Standard, *args, **kwargs).
----------------------------	----------------	---	---

Topic	Syntax	Real-Time Architectural Use	Interview Cross-Question
Multithreading	threading	I/O Bound: Parallelizing 50 API calls to fetch weather data.	"Why doesn't it speed up math?" (Ans: The GIL prevents true CPU parallelism).
Multiprocessing	multiprocessing	CPU Bound: Parallelizing heavy regex parsing or data encryption on multiple cores.	"What is the main cost of multiprocessing?" (Ans: IPC - Inter-Process Communication overhead).

Topic	Concept	Architectural "Why"	Interview Cross-Question
-------	---------	---------------------	--------------------------

Lazy Eval	Transformations	Optimization: Spark builds a DAG and optimizes the execution plan before moving data.	"Name an Action that triggers execution." (Ans: collect(), save(), show()).
Repartition	repartition(n)	Full Shuffle: Use when increasing partitions or balancing data for a join.	"Why is coalesce better for decreasing?" (Ans: It avoids a full shuffle by moving data locally).
Caching	df.cache()	Performance: Use when a middle-layer DF is used in 3+ downstream aggregations.	"Difference between cache() and persist()?" (Ans: Cache is memory only; Persist allows Disk/S3 levels).

*args and kwargs are the keys to building **flexible, reusable interfaces**. Instead of hard-coding every parameter, these allow you to design "Contract-First" functions that can adapt to changing data schemas without breaking.

1. *args (Non-Keyword Arguments)

The Concept: It allows a function to accept any number of **positional** arguments. Inside the function, args is a **Tuple**.

- **Syntax:** def function_name(*args):
- **Real-Time DE Use Case: Data Consolidation.** Suppose you are writing a utility to merge multiple CSV file paths into a single dataframe. You don't know if the user will provide 2

files or 20.

- **Example:**

Python

```
def merge_files(*file_paths):
    for path in file_paths:
        print(f"Ingesting: {path}")
        # Logic to read and append data...

merge_files("s3://bucket/jan.csv", "s3://bucket/feb.csv")
```

2. kwargs (Keyword Arguments)

The Concept: It allows a function to accept any number of **named** arguments (key-value pairs). Inside the function, kwargs is a **Dictionary**.

- **Syntax:** def function_name(kwargs):
- **Real-Time DE Use Case: Metadata & Configuration.** When building a generic database connector, different databases (Postgres, MySQL, Snowflake) require different parameters (host, port, warehouse, schema).
- **Example:**

Python

```
def connect_to_db(**config):
    if config.get("type") == "snowflake":
        print(f"Connecting to {config['warehouse']}...")
        # Logic to initialize connection...

connect_to_db(type="snowflake",
warehouse="COMPUTE_WH", user="Mantu")
```

3. Architect's View: Why use them together?

In production frameworks, you often see def func(*args, kwargs). This is the "**Universal Wrapper**" pattern. It's used heavily in **Decorators** because a decorator must be able to wrap *any* function, regardless of its original signature.

4. Interview "Cross-Questions" (Pressure Test)

- **Q: Can I change the names 'args' and 'kwargs'?**
 - *Ans:* Yes. The magic is in the asterisks (* and), not the

names. However, args and kwargs are the industry standard for readability.

- **Q: What is the mandatory order of parameters?**
 - *Ans:* 1. Standard arguments, 2. *args, 3. kwargs. If you swap them, Python will raise a SyntaxError.
- **Q: How do you "unpack" a list into a function using these?**
 - *Ans:* If you have a list `my_list = [1, 2]`, you can pass it to a function as `func(*my_list)`. This "unpacks" the list into individual positional arguments.
- **Q: Why are these preferred over passing a single List or Dict?**
 - *Ans:* It provides a cleaner API for the end-user. It feels more "Pythonic" to call `func(a, b, c)` than `func([a, b, c])`.

As a Data Engineering Architect, I've categorized these frequently asked topics into **Execution Layers**. In an interview, you should explain these not just as syntax, but as tools to solve specific engineering bottlenecks like **Memory Pressure**, **I/O Latency**, and **System Scalability**.

1. The Memory Efficiency Layer

These topics are about processing data that is larger than your available RAM.

Topic	Architect's "Why"	Real-Time Example
Generators	Achieves Lazy Evaluation . It prevents \$OOM\$ (Out of Memory) errors by streaming data.	Processing a 100GB CSV file line-by-line instead of loading it as a list.

Memory Optimization	Focuses on reducing the footprint of objects (e.g., using <code>__slots__</code> or choosing Tuples over Lists for fixed data).	Converting a large list of dictionaries into a list of tuples to save space during a migration.
File Handling	Efficiently managing resources using Context Managers (with statement) to ensure files are closed and memory is flushed.	Reading multi-gigabyte log files using a generator to extract specific error codes.

2. The Performance & Concurrency Layer

How to make your code run faster by utilizing hardware effectively.

Topic	Architect's "Why"	Real-Time Example
Multithreading	Best for I/O-bound tasks. It allows the CPU to switch tasks while waiting for data.	Fetching data from 50 different REST API endpoints simultaneously.
Lambda/Map/Filter	Provides a concise, functional way to apply transformations across datasets.	Using <code>filter()</code> to remove corrupted records from a raw JSON stream before ingestion.

3. The Architecture & Clean Code Layer

How to build maintainable, enterprise-grade frameworks.

Topic	Architect's "Why"	Real-Time Example
-------	-------------------	-------------------

Decorators	Implements Separation of Concerns . It wraps core logic with operational logic like retries or logging.	Attaching an @retry decorator to a function that uploads files to S3 to handle intermittent network flickers.
Exception Handling	Ensures Pipeline Robustness . It prevents a single "bad" record from crashing a 10-hour job.	Using try-except-finally to ensure a database connection is closed even if the data transformation fails.
OOP Basics	Encourages Reusability . Using Inheritance to create a base "Extractor" class that specific "S3" or "SFTP" classes extend.	Creating a DataSink interface that different warehouse connectors (Snowflake, Redshift) must implement.

4. The Distributed Computing Layer (Big Data)

Processing at the Petabyte scale where single-machine Python fails.

Topic	Architect's "Why"	Real-Time Example
PySpark Internals	Utilizes Distributed Execution and the Catalyst Optimizer to process data across a cluster.	Transforming TBs of clickstream data using DAG optimization to minimize shuffles.
API Ingestion	The entry point for data. Requires handling rate limits, authentication, and pagination.	Building a Python wrapper to pull daily sales data from Shopify and push it to a Bronze layer.



Potential Architect-Level Interview Questions:

1. **Generators vs. Multithreading:** "If you have a 50GB file,

would you use a Generator or Multithreading to process it?"

- *Ans:* A **Generator** handles the memory constraint, while **Multithreading** might speed up the I/O if the file is being read from a network drive (S3), but won't help if the bottleneck is CPU-bound.

2. **PySpark vs. Python:** "When does it become more cost-effective to switch from a Python script to a PySpark cluster?"

- *Ans:* When the dataset size exceeds the memory of the largest available single machine, or when the transformation logic is complex enough to benefit from distributed parallelization.

sort() vs sorted():

GOLDEN MEMORY RULE

👉 `sort()` changes existing list.

👉 `sorted()` creates new sorted result.

Feature	<code>sort()</code>	<code>sorted()</code>
Original modified	✓	✗
Returns new object	✗	✓
Works on list only	✓	✗
Works on tuple/set/string	✗	✓
Memory efficient	✓	✗

✓ *Python Strings & File Handling*

Strings and files are essential for working with text and data in Python. Let's dive into how to manipulate strings and read/write files!

◆ *1. Strings*

Strings are sequences of characters. You can create them with quotes and manipulate them using built-in methods.

Example:

```
Awaa```  
name = "Pythonista"
```

```
print(name.upper())    # PYTHONISTA  
print(name.lower())    # pythonista  
print(name[::-1])      # atsinohtyP  
```\
```

**\*Common Methods:\***

- ``len(s)`` → length of string
- ``s.strip()`` → removes whitespace
- ``s.replace("a", "b")`` → replaces characters
- ``s.split(",")`` → splits into list.
- ``s.find("text")`` → finds position

**\*Challenge: Palindrome Checker\***

```
```python  
word = input("Enter a word: ")  
  
if word == word[::-1]:  
    print("It's a palindrome!")  
else:  
    print("Not a palindrome.")  
```\
```

## ◆ **\*2. String Formatting\***

Use ``f-strings`` to embed variables in strings.

**\*Example:\***

```
```python  
name = "Alex"  
age = 25  
print(f"{name} is {age} years old.")  
```\
```

**\*Challenge: Personalized Greeting\***

```
```python  
user = input("What's your name? ")  
print(f"Hello, {user}! Welcome to Python.")  
```\
```

### ◆ \*3. File Handling: Read & Write\*

Python can read from and write to files using ``open()``.

#### **\*Reading a File:\***

```
```python
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```
```

#### **\*Writing to a File:\***

```
```python
with open("output.txt", "w") as file:
    file.write("Hello, file world!")
```
```

#### **\*Challenge: Save User Notes\***

```
```python
note = input("Write your note: ")

with open("notes.txt", "a") as file:
    file.write(note + "\n")

print("Note saved!")
```
```

### ◆ \*4. Mini Project: Word Counter

#### **\*Goal:\***

- Ask user for a filename
- Count number of words in the file
- Case-insensitive counting
- Display the result
- Count frequency of each word

#### **\*Python Code:\***

```
```python
filename = input("Enter filename: ")

try:
    with open(filename, "r") as file:
        text = file.read()
```

```

words = text.lower().split()
print(f"Word count: {len(words)}")

except FileNotFoundError:
    print("File not found.")

```

#Count frequency of each word

```

from collections import Counter

word_count = Counter(words)
print(word_count)

```



*Top 10 Python List Methods You Must Know

```


```

```

lst = [1, 2, 3]

```

```


```

*1. `append()` – *Add element at the end as an **ITEM**.

› `lst.append(5)` → `[1, 2, 3, 5]`

*2. `extend()` – *Add multiple elements* —**ITERATION**.

› `lst.extend([6, 7])` → `[1, 2, 3, 5, 6, 7]`

*3. `insert()` – *Insert at specific position*

› `lst.insert(1, 'a')` → `[1, 'a', 2, 3, 5, 6, 7]`

› `lst.insert(-1, 'a')` → `[1, 'a', 2, 3, 5, 6, 'a', 7]`

*4. `remove()` – *Remove first occurrence of value*

› `lst.remove(3)` → `[1, 'a', 2, 5, 6, 7]`

*5. `pop()` – *Remove & return item at index*

› `lst.pop()` → returns last item and removes it

*6. `index()` – *Find index of a value*

› `lst.index(2)` → `2`

*7. `count()` – *Count occurrences of value*

> `lst.count(1)` → `1`

*8. `sort()` – *Sort list in-place*

> `lst.sort()` → modifies the original list

9. `sorted()` – *Return a new sorted list*

> `new_lst = sorted(lst)` → creates a new sorted list without modifying original list

*10. `reverse()` – *Reverse list in-place*

> `lst.reverse()` → modifies the original list

*11. `copy()` – *Make shallow copy of list*

> `new_lst = lst.copy()`

💡 *Bonus Tip:* Use `sorted(lst)` for a sorted version without changing the original.



*Top 10 Python String Methods You Should Know

```

`s = "hello world"`

```

*1. `upper()` – *Convert to uppercase*

> `s.upper()` → `"HELLO WORLD"`

*2. `lower()` – *Convert to lowercase*

> `s.lower()` → `"hello world"`

*3. `capitalize()` – *Capitalize first letter*

> `s.capitalize()` → `"Hello world"`

*4. `title()` – *Capitalize first letter of each word*

> `s.title()` → `"Hello World"`

*5. `strip()` – *Remove leading/trailing spaces*

> `" hello ".strip()` → `"hello"`

*6. `replace()` – *Replace substring*

> `s.replace("world", "Python")` → `"hello Python"``

7. `split()`` – *Split string into list*

> `s.split()` → `["hello", "world"]``

8. `join()`` – *Join list into string*

> `" ".join(["Python", "is", "fun"])` → `"Python is fun"``

9. `find()`` – *Find first occurrence*

> `s.find("o")` → `4``

10. `startswith()`` / `endswith()`` – *Check prefix/suffix*

> `s.startswith("hello")` → `True``

> `s.endswith("world")` → `True``

💡 *Bonus Tip:* Use ``in`` to check if a substring exists → ``"Python" in s``



*Top 10 Python Set Methods You Should Know

`a = {1, 2, 3}`

`b = {3, 4, 5}`

1. `add()`` – *Add an element*

> `a.add(4)` → `{1, 2, 3, 4}``

2. `update()`` – *Add multiple elements*

> `a.update([5, 6])` → `{1, 2, 3, 5, 6}``

3. `remove()`` – *Remove element (error if not found)*

> `a.remove(2)``

4. `discard()`` – *Remove element (no error if not found)*

> `a.discard(100)` → No error`

5. `pop()`` – *Remove and return a random element*

> `a.pop()` → Removes any element`

6. `clear()`` – *Remove all elements*

> `a.clear()` → `set()``

*7. `union()` - *Combine sets (no duplicates)*

> `a.union(b)` → `{1, 2, 3, 4, 5}`

*8. `intersection()` - *Common elements*

> `a.intersection(b)` → `{3}`

*9. `difference()` - *Elements in `a` but not in `b`*

> `a.difference(b)` → `{1, 2}`

*10. `issubset()` / `issuperset()` - *Check relations*

> `{1, 2}.issubset(a)` → `True`

> `a.issuperset({1})` → `True`



Bonus Tip: Sets automatically remove duplicates →

`set([1, 1, 2, 3])` → `{1, 2, 3}`



Top 10 Python Dictionary Methods You Should Know

`data = {'a': 1, 'b': 2, 'c': 3}`

*1. `get()` - *Safely get value*

> `data.get('b')` → `2`

> `data.get('x')` → `None`

*2. `keys()` - *Get all keys*

> `data.keys()` → `dict_keys(['a', 'b', 'c'])`

*3. `values()` - *Get all values*

> `data.values()` → `dict_values([1, 2, 3])`

*4. `items()` - *Get key-value pairs*

> `data.items()` → `dict_items([('a', 1), ...])`

*5. `update()` - *Merge another dict*

> `data.update({'d': 4})` → `{'a':1, ..., 'd':4}`

*6. `pop()` - *Remove by key*

> `data.pop('b')` → `2`

*7. `popitem()` – *Remove last inserted pair*

> `data.popitem()` → `('c', 3)`

*8. `clear()` – *Remove all items*

> `data.clear()` → `{}`

*9. `setdefault()` – *Get value or set if not present*

> `data.setdefault('e', 0)` → Adds `'e': 0` if missing

*10. `fromkeys()` – *Create dict from keys*

> `dict.fromkeys(['x', 'y'], 0)` → `{'x': 0, 'y': 0}`



Dicts in Python 3.7+ preserve insertion order!



*Top 10 Python Tuple Operations You Must Know

```

tup = (1, 2, 3, 4, 2)

```

*1. `len()` – *Get length of tuple*

> `len(tup)` → `5`

*2. `count()` – *Count occurrences of a value*

> `tup.count(2)` → `2`

*3. `index()` – *Find index of a value*

> `tup.index(3)` → `2`

*4. `in` keyword – *Check existence*

> `2 in tup` → `True`

*5. `for` loop – *Iterate through elements*

> `for i in tup: print(i)`

*6. `slicing` – *Access sub-parts*

> `tup[1:4]` → `(2, 3, 4)`

*7. `+` operator – *Concatenate tuples*

> `tup + (5, 6)` → `(1, 2, 3, 4, 2, 5, 6)`

8. ``*`` operator – *Repeat tuple*

› ``tup * 2`` → ``(1, 2, 3, 4, 2, 1, 2, 3, 4, 2)``

*9. ``tuple()`` – *Convert iterable to tuple*

› ``tuple([7, 8])`` → ``(7, 8)``

*10. ``min()`` / ``max()`` – *Get smallest/largest item*

› ``min(tup)`` → ``1``

› ``max(tup)`` → ``4``

💡 *Bonus:* Tuples are *immutable* – they can't be changed after creation.

*Top 10 Python Built-in Functions You Should Know

1. *Basic Syntax:*

› `[x * 2 for x in nums]` → `[2, 4, 6, 8, 10]`

2. *Filter Even Numbers:*

› `[x for x in nums if x % 2 == 0]` → `[2, 4]`

3. *Filter Odd Numbers:*

› `[x for x in nums if x % 2 != 0]` → `[1, 3, 5]`

4. *Square Numbers:*

› `[x**2 for x in nums]` → `[1, 4, 9, 16, 25]`

5. *Cube Numbers:*

› `[x**3 for x in nums]` → `[1, 8, 27, 64, 125]`

6. *Replace Values Using if-else:*

› `[x if x > 2 else 0 for x in nums]` → `[0, 0, 3, 4, 5]`

7. *Label Even/Odd:*

› `['even' if x%2==0 else 'odd' for x in nums]`

→ `['odd', 'even', 'odd', 'even', 'odd']`

8. **Convert to String:**

› `[str(x) for x in nums]` → `['1', '2', '3', '4', '5']`

9. **Get First Letter:**

```
names = ['Ravi', 'Anita', 'Kumar']
```

› [x[0] for x in names] → ['R', 'A', 'K']

10. Uppercase Strings:

› [x.upper() for x in names] → ['RAVI', 'ANITA', 'KUMAR']

11. Lowercase Strings:

› [x.lower() for x in names] → ['ravi', 'anita', 'kumar']

12. String Length:

› [len(x) for x in names] → [4, 5, 5]

13. Flatten 2D List:

```
matrix = [[1,2], [3,4]]
```

› [x for row in matrix for x in row] → [1, 2, 3, 4]

14. Nested Loops:

› [(x,y) for x in [1,2] for y in ['a','b']]
→ [(1,'a'), (1,'b'), (2,'a'), (2,'b')]

15. Remove NULL/Empty Values:

```
vals = [1, '', None, 5]
```

› [x for x in vals if x] → [1, 5]

16. Numbers Greater Than 3:

› [x for x in nums if x > 3] → [4, 5]

17. Reverse Strings:

› [x[::-1] for x in names] → ['ivaR', 'atinA', 'ramuK']

18. Create Tuple List:

› [(x, x**2) for x in nums]
→ [(1,1), (2,4), (3,9), (4,16), (5,25)]

19. Remove Duplicates Using Set:

```
nums = [1,1,2,3,3,4]
```

› [x for x in set(nums)] → [1, 2, 3, 4]

20. *Multiple Conditions:*

› [x for x in nums if x > 1 if x < 4] → [2, 3]

***Python Dictionary Comprehensions!**

Dictionary comprehensions let you create or transform dictionaries in a single, readable line.

```
...
```

```
nums = [1, 2, 3, 4]
```

```
...
```

1. Basic Syntax:

› `{key: value for item in iterable}`

› `{x: x\*\*2 for x in nums}` → `{1: 1, 2: 4, 3: 9, 4: 16}`

2. With Condition:

› `{x: x\*\*2 for x in nums if x % 2 == 0}` → `{2: 4, 4: 16}`

(*Only even numbers included*)

3. With If-Else:

› `{x: 'even' if x % 2 == 0 else 'odd' for x in nums}`

→ `{1: 'odd', 2: 'even', 3: 'odd', 4: 'even'}`

4. From Two Lists:

```
...
```

```
keys = ['a', 'b']
```

```
values = [1, 2]
```

```
{ k: v for k, v in zip(keys, values) } → {'a': 1, 'b': 2}
```

```
...
```

5. Swap Keys & Values:

```
...
```

```
d = {'a': 1, 'b': 2}
```

```
{v: k for k, v in d.items()} → {1: 'a', 2: 'b'}
```

```
...
```

6. Nested Loop:

```
> `{(x, y): x * y for x in range(2) for y in range(2)}`  
→ `{(0, 0): 0, (0, 1): 0, (1, 0): 0, (1, 1): 1}`
```



Great for quick transformations, filtering, and clean code!



*Python Set Comprehensions!

Set comprehensions help you create sets quickly and cleanly in one line. Sets store *unique* items, no duplicates.

```
...
```

```
nums = [1, 2, 2, 3, 4, 4, 5]
```

```
...
```

1. Basic Syntax:

```
> `{expression for item in iterable}`  
> `{x * 2 for x in nums}` → `{2, 4, 6, 8, 10}`
```

2. With Condition:

```
> `{x for x in nums if x % 2 == 0}` → `{2, 4}`  
(*Only even numbers*)
```

3. With `if-else` inside:

```
> `{x if x > 2 else 0 for x in nums}` → `{0, 3, 4, 5}`
```

4. Nested Loops:

```
> `{(x, y) for x in [1,2] for y in ['a','b']}` → `{(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')}`
```

5. Remove duplicates from list:

```
> `{x for x in nums}` → `{1, 2, 3, 4, 5}`
```



Sets keep only unique values, so great for removing duplicates fast!



File Handling & CSV Operations in Python



Interviewer:

Can you explain how file handling works in Python?

 *Candidate:*

Yes! We use `open()` to read/write files.

Syntax:

```
`file = open("filename.txt", "mode")`
```

Modes:

'r' → Read

'w' → Write (overwrites)

'a' → Append

'r+' → Read & write

'b' → Binary mode

Use `file.close()` or better:

```
```python
with open("data.txt", "r") as f:
 content = f.read()
 print(content)
```
```

 *Interviewer:*

What's the difference between `read()`, `readline()`, and `readlines()`?

 *Candidate:*

read() → Entire file as string

readline() → One line

readlines() → List of lines

Example:

```
```python
with open("demo.txt", "r") as f:
 print(f.read())
 print(f.readline())
 print(f.readlines())
```
```

 *Interviewer:*

How do you write data into a file?

 *Candidate:*

Use `write()` or `writelines()`

```
```python
```

```
with open("output.txt", "w") as f:
 f.write("Python is great!\n")
 f.writelines(["Data\n", "Analytics\n", "Interview\n"])
...
```



\*Interviewer:\*

**What happens if you open a file in write mode that already exists?**



\*Candidate:\*

It overwrites the file. Use `'a'` to append:

```
```python
with open("output.txt", "a") as f:
    f.write("Appending new line.\n")
...
```



Interviewer:

What if the file doesn't exist and we try to read it?



Candidate:

It raises `FileNotFoundError`. Use try-except:

```
```python
try:
 with open("missing.txt", "r") as f:
 print(f.read())
except FileNotFoundError:
 print("File not found!")
...
```



\*Interviewer:\*

**How do you handle CSV files in Python?**



\*Candidate:\*

Using `'csv'` module:

```
```python
import csv
```

with open("data.csv", "r") as file:

```
    reader = csv.reader(file)
```

```
    for row in reader:
```

```
        print(row)
```

with open("data.csv", "w", newline="") as file:

```
writer = csv.writer(file)
writer.writerow(["Name", "Age"])
writer.writerow(["Deepak", 27])
...
```



Interviewer:

How to read a CSV using pandas?



Candidate:

```
```python
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())

df.to_csv("new_data.csv", index=False)
```
```



Interviewer:

How to handle large CSV files? **VVI**



Candidate:

Use `chunksize` in pandas:

```
```python
for chunk in pd.read_csv("large_file.csv", chunksize=10000):
 print(chunk.shape)
```
```



Interviewer:

How to check if a file exists?



Candidate:

```
```python
import os

if os.path.exists("data.csv"):
 print("File found!")
else:
 print("File not found!")
```
```



Mini Practice Questions

✅ *Q1. Count lines in a text file*

```
```python
with open("sample.txt", "r") as f:
 lines = f.readlines()
print(len(lines))
```
```

✅ *Q2. Copy contents from one file to another* VVi

```
```python
with open("source.txt", "r") as src, open("target.txt", "w") as tgt:
 tgt.write(src.read())
```
```

✅ *Q3. Print names of people above 25 from CSV*

```
```python
import csv

with open("people.csv", "r") as f:
 reader = csv.DictReader(f)
 for row in reader:
 if int(row["Age"]) > 25:
 print(row["Name"])
```
```

✅ *Q4. Find average of a column using pandas*

```
```python
import pandas as pd
df = pd.read_csv("sales.csv")
print(df["Revenue"].mean())
```
```

✅ *Q5. Append user input to a file*

```
```python
text = input("Enter text: ")
with open("notes.txt", "a") as f:
 f.write(text + "\n")
```
```

Python (including topics like:

- finding duplicates in a list
- dictionary comprehension
- finding duplicate characters in a string)

=====

1) List files in a directory. VVI

```
import os
files = os.listdir("data/raw/")
print(files)
```

This returns all filenames in the folder.

2) Filter only CSV files

```
csv_files = [
    f for f in os.listdir("data/raw/")
    if f.endswith(".csv")
]
```

Common in ingestion pipelines where folders contain mixed files.

3) Build full file paths

```
import os
folder = "data/raw/"

for file in csv_files:
    full_path = os.path.join(folder, file)
    print(full_path)
```

Correct path handling is critical — especially across environments.

4) The cleaner approach: pathlib

```
from pathlib import Path
raw_path = Path("data/raw")

for file in raw_path.glob("*.csv"):
    print(file)
```

Why many engineers prefer pathlib:

- * cleaner syntax
- * OS-independent paths
- * built-in pattern matching

5) Where directory operations are used

You'll see this pattern when:

- processing daily ingestion folders
- looping through vendor drop files
- merging multiple CSVs
- scanning cloud storage paths
- building partition-based pipelines